

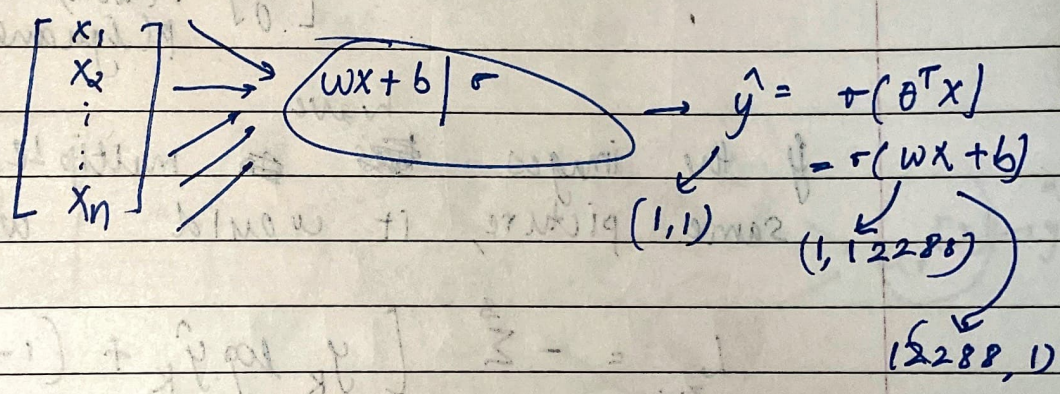
Lecture - 11 - Introduction to Neural Networks & Lecture 12 - Backprop & Improving neural networks. Deep Learning

- computational power
- data available
- algorithms

Goal: Find cats in images } $1 \rightarrow$ presence of cat
 $0 \rightarrow$ absence of cat

A image in computer science can be represented as $64 \times 64 \times 3$ image, the 3 is for R, G, B.

so the $64 \times 64 \times 3 \rightarrow$ this is converted to a column vector.



- Initiate w, b
- Find the optimal w, b
- Use $\hat{y} = \sigma(w x + b)$ to predict.

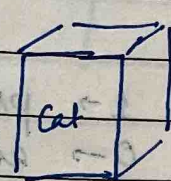
$$L = - [y \log \hat{y} + (1-y) \log (1-\hat{y})]$$

$$w = w - \alpha \frac{\partial L}{\partial w}$$

$$b = b - \alpha \frac{\partial L}{\partial b}$$

Eq. 1) Neuron = linear + activation
Eq. 2) Model = architecture + parameters

Goal 2.0 : Find cat / lion / iguana in images


 $= \begin{pmatrix} | \\ | \\ | \end{pmatrix} \begin{matrix} a_1^{(1)} \\ a_2^{(1)} \\ a_3^{(1)} \end{matrix}$
 $\hat{y}_1 = a_1^{(1)} = \sigma(w_1^{(1)}x + b_1^{(1)})$
 $\hat{y}_2 = a_2^{(1)} = \sigma(w_2^{(1)}x + b_2^{(1)})$
 $\hat{y}_3 = a_3^{(1)} = \sigma(w_3^{(1)}x + b_3^{(1)})$

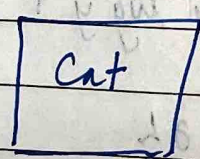
For cat, the output label would be 

$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$
 $\begin{matrix} p(\text{cat}) \\ p(\text{lion}) \\ p(\text{iguana}) \end{matrix}$

If the images ~~has~~ ~~are~~ have multiple animals in same picture, it would work as well.

$$L_{3w} = - \sum_{k=1}^3 [y_k \log \hat{y}_k + (1 - y_k) \log (1 - \hat{y}_k)]$$

Goal 3.0 + constraint
unique animal on an image


 $= \begin{pmatrix} x_1 \\ \vdots \\ x_N \end{pmatrix}$
 $\rightarrow z_1^{(1)} = \frac{e^{z_1^{(1)}}}{\sum_{k=1}^3 e^{z_k^{(1)}}}$
 $\rightarrow z_2^{(1)} = \frac{e^{z_2^{(1)}}}{\sum_{k=1}^3 e^{z_k^{(1)}}}$

This is softmax multi-class regression.

The sum of probabilities in softmax is 1.

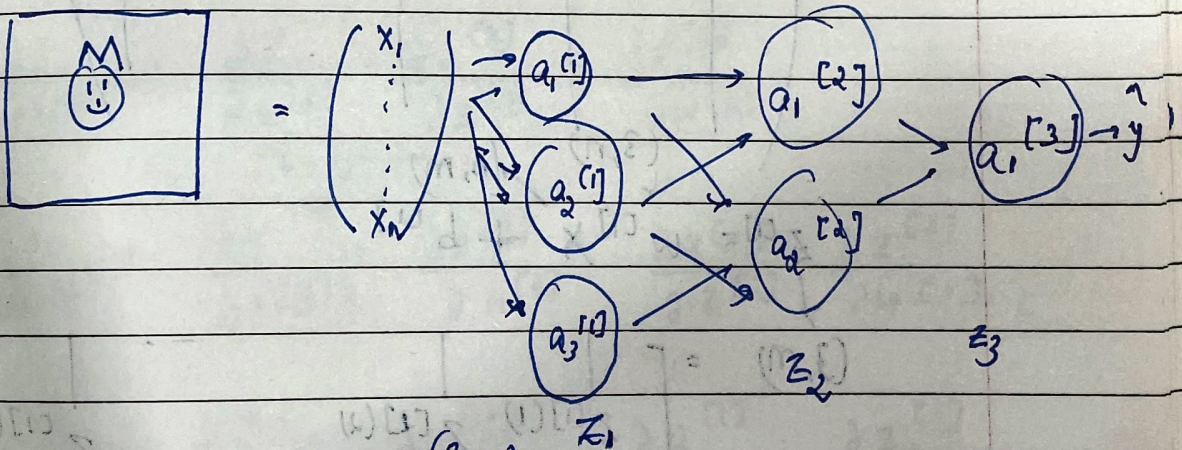
$$\text{Cat} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \text{Lion} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \text{Iguana} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

for multiclass classification problems, the cross entropy loss function can be used.

$$L_{CE} = - \sum_{k=1}^3 y_k \log \hat{y}_k$$

Neural Networks

Goal images $\Rightarrow$ cat vs no cat
 (1) (0)



Propagation equations

$$z^{[1]} = w^{[1]} x + b^{[1]}$$

(3,1)

$$a^{[1]} = \sigma(z_1^{[1]})$$

(3,1)

$$z^{[2]}_{(2,1)} = w^{[2]}_{(2,1)} a^{[1]}_{(1,1)} + b^{[2]}_{(2,1)}$$

$$a^{[2]}_{(2,1)} = \sigma(z^{[2]}_{(2,1)})$$

$$z^{[3]}_{(1,1)} = w^{[3]}_{(1,1)} a^{[2]}_{(1,1)} + b^{[3]}_{(1,1)}$$

$$a^{[3]}_{(1,1)} = \sigma(z^{[3]}_{(1,1)})$$

Q. What happens when we have a input batch of m examples.

$$X = \begin{pmatrix} | & | & | & | \\ \nearrow id & \nearrow id & & \\ x^{(1)} & x^{(2)} & x^{(3)} & x^{(m)} \\ | & | & | & | \end{pmatrix}$$

$$z^{[1]}_{(3,m)} = w^{[1]}_{(3,m)} X + b^{[1]}_{(3,m)}$$

$$(3,m) = \begin{bmatrix} | & | & | \\ z^{[1]}_{(1)}(1) & z^{[1]}_{(1)}(2) & z^{[1]}_{(1)}(m) \\ | & | & | \\ | & | & | \end{bmatrix}$$

The parameters of $w^{[1]}$ should remain the same.

But $b^{(1)}$ remains as $(3, 1)$. We would have to use ~~the~~ broadcasting technique here to take care of $b^{(1)}$.

Define loss/cost function

$$J(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m L^{(i)}$$

(with $L^{(i)} = -[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)})]$)

Back propagation Algorithm

$$\forall l=1 \dots 3 \quad w^{[l]} = w^{[l]} - \alpha \frac{\partial J}{\partial w^{[l]}}$$

$$b^{[l]} = b^{[l]} - \alpha \frac{\partial J}{\partial b^{[l]}}$$

$$\frac{\partial J}{\partial w^{[3]}} = \frac{\partial J}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial w^{[3]}}$$

$$\frac{\partial J}{\partial w^{[2]}} = \frac{\partial J}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

$$\frac{\partial J}{\partial w^{[1]}} = \frac{\partial J}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

computing

Instead of $\frac{\partial J}{\partial w^{[3]}}$, we will compute

$$\frac{\partial L}{\partial w^{[3]}} = - \left[y^{(i)} \frac{\partial}{\partial w^{[3]}} \log \sigma(w^{[3]} a^{[2]} + b^{[3]}) + (1 - y^{(i)}) \frac{\partial}{\partial w^{[3]}} \log (1 - \sigma(w^{[3]} a^{[2]} + b^{[3]})) \right]$$

$$\begin{aligned} \frac{\partial L}{\partial w^{[3]}} &= - \left[y^{(i)} \cdot \frac{1}{a^{[3]}} a^{[3]} (1 - a^{[3]}) a^{[2]T} + \right. \\ &\quad \left. (1 - y^{(i)}) \cdot \frac{1}{(1 - a^{[3]})} (-1) \cdot a^{[3]} (1 - a^{[3]}) a^{[2]T} \right] \\ &= - \left[y^{(i)} (1 - a^{[3]}) a^{[2]T} - (1 - y^{(i)}) a^{[3]} a^{[2]T} \right] \end{aligned}$$

$$\frac{\partial L}{\partial w^{[3]}} = - (y^{(i)} - a^{[3]}) a^{[2]T}$$

$$\frac{\partial J}{\partial w^{[3]}} = - \frac{1}{M} \sum_{i=1}^M (y^{(i)} - a^{[3]}) a^{[2]T}$$

moving forward with backpropagation -

$$\begin{aligned} \frac{\partial L}{\partial w^{[2]}} &= \frac{\partial L}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial z^{[3]}} \cdot \frac{\partial z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \left(\frac{\partial z^{[2]}}{\partial w^{[2]}} \right) \\ &\quad \downarrow \qquad \qquad \qquad \downarrow \qquad \qquad \qquad \downarrow \qquad \qquad \qquad \downarrow \\ &= - (y^{(i)} - a^{[3]}) \cdot w^{[3]T} \cdot a^{[2]} \cdot (1 - a^{[2]}) \end{aligned}$$

$$\frac{\partial L}{\partial w^{[2]}} = \underbrace{(a^{[3]} - y)}_{(1,1)} \underbrace{w^{[3]T}}_{(2,1)} + \underbrace{a^{[2]}(1 - a^{[2]})}_{(2,1)} \underbrace{a^{[1]T}}_{(1,3)}$$

element wise

Re-arranging these terms.

$$\frac{\partial L}{\partial w^{[2]}} = \underbrace{w^{[3]T}}_{(2,1)} + \underbrace{a^{[2]}(1 - a^{[2]})}_{(2,1)} \underbrace{(a^{[3]} - y)}_{(1,3)} \underbrace{a^{[1]T}}_{(1,3)}$$

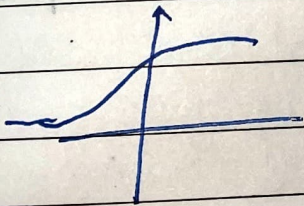
element wise

(2,3)

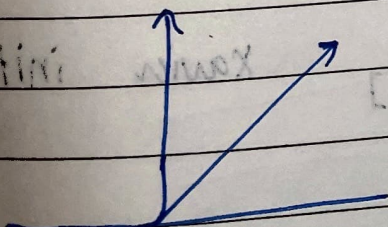
Improving Neural networks

Sigmoid $\sigma(z) = \frac{1}{1 + e^{-z}}$

$$\sigma'(z) = \sigma(z) * (1 - \sigma(z))$$



$$\text{ReLU}(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{if } z > 0 \end{cases}$$

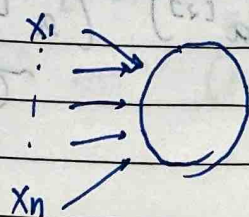


$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - \tanh(z)^2$$

Vanishing / Exploding gradients

Example with 1 neuron $\rightarrow$



$$a = \sigma(z)$$

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

large $n \rightarrow$ small w_i
 $w_i \sim \frac{1}{n}$

Initialization methods used commonly
 For sigmoid

$$w^{[1]} = \text{np.random.randn}(\text{shape}) * \text{np.sqrt}\left(\frac{1}{n^{[1]}}\right)$$

For relu

$$w^{[1]} = \text{np.random.randn}(\text{shape}) * \text{np.sqrt}\left(\frac{2}{n^{[1]}}\right)$$

For tanh

$$w^{[1]} = \sqrt{\frac{1}{n^{[1]}}}$$

Xavier initialization.

(c) Optimization

$$X = (x^{(1)} \ x^{(2)} \ \dots \ x^{(m)})$$

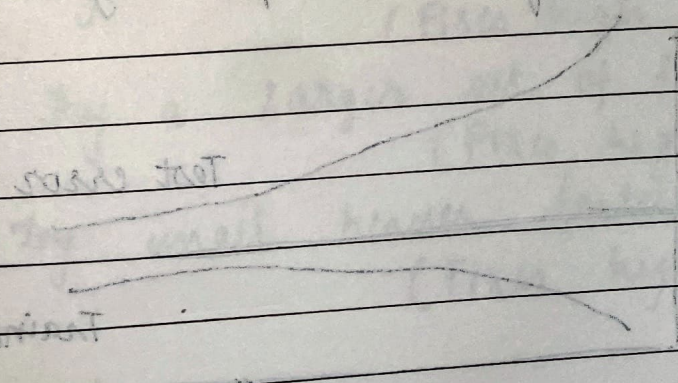
$$Y = (y^{(1)} \ y^{(2)} \ \dots \ y^{(m)})$$

$$X = \begin{pmatrix} x^{(1)} & x^{(2)} \end{pmatrix}$$

$$Y = \begin{pmatrix} y^{(1)} & y^{(2)} \end{pmatrix}$$

Algorithm: For iteration $t=1 \dots$
 select batch $(x^{(t)}, y^{(t)})$
 Forward prop.
 Back prop.
 update $w^{[1]}, b^{[1]}$

$$J = \frac{1}{1000} \sum_{i=1}^{1000} \ell^{(i)}$$



we trained set size